



BRENT OZAR
UNLIMITED®

Measuring Performance and Adjusting Instance Sizes

Deck 5 p1

Agenda

Reservations, and why we don't do 'em right away

How Alex, Billie, and Elliot use reservations

The go-live plan: how to adjust performance

Monitoring wait stats to determine adjustments



Deck 5 p2

The cloud defaults to flexibility.

In the cloud, you can:

Get any server type you want in seconds

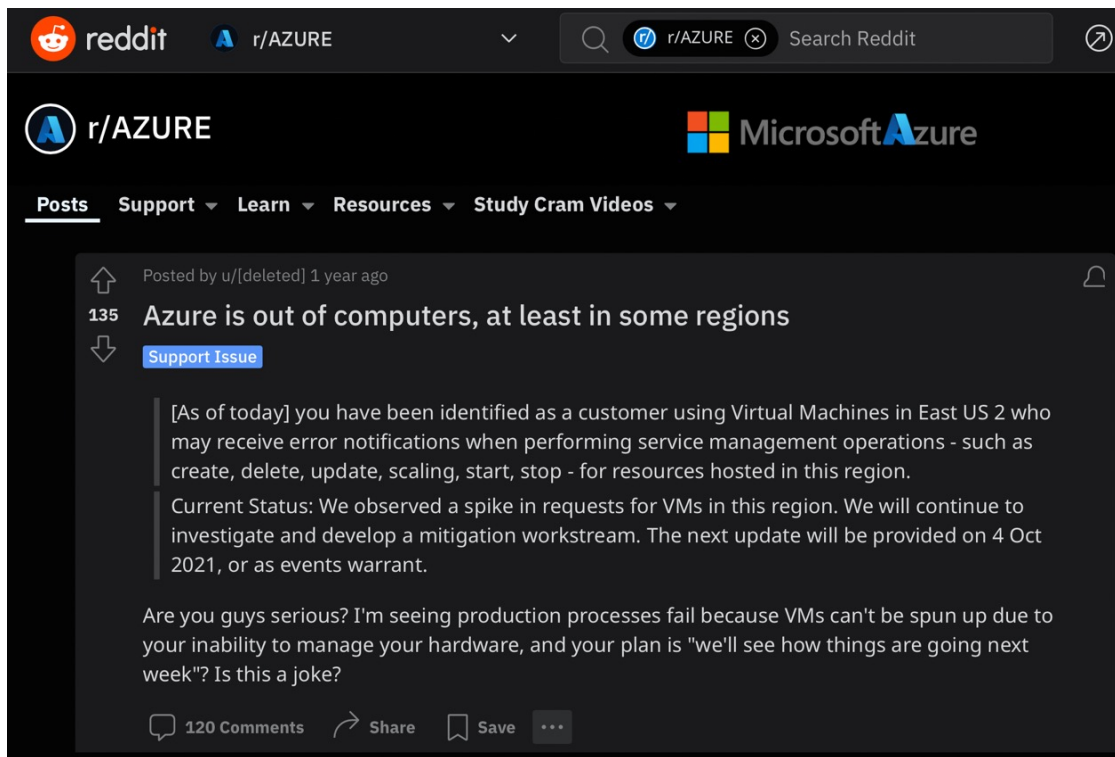
Get as many of them as you want

In any existing data center you want

With some exceptions:

https://www.reddit.com/r/AZURE/comments/py4pdu/azure_is_out_of_computers_at_least_in_some_regions/





So when I do cloud migrations:

“We’ll work hard to design, build, and test the right overall **architecture** for the **RPO/RT0 you require.**”

“Then, once we’re live, we’ll work hard to measure performance to **tweak sizes** for the **performance you want.**”

“When you’re happy with both, you can lock down savings with reservations.”



Businesses want exact numbers.

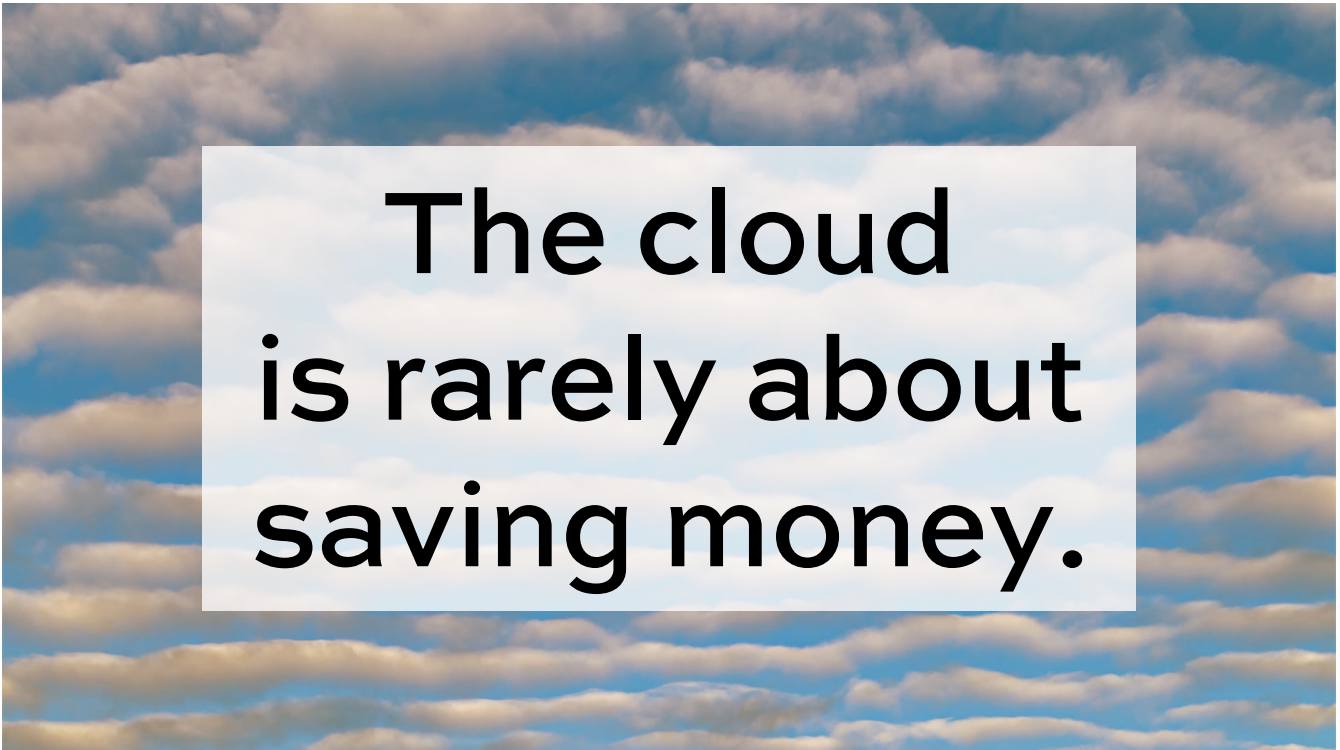
Managers want to predict exactly how much their monthly bills will be in the cloud.

You can give them numbers.

They'll just likely be wrong because:

- It's hardware & storage that's new to you
- App servers & designs often change too
- Network ingress & egress charges are unpredictable if some stuff is still on-premises

Deck 5 p7



The cloud
is rarely about
saving money.

It CAN save you money...

If your requirements go up/down **frequently**:

- Companies that process data 1 day per month
- Companies that have 2-3 big sale days per year

Or if your requirements stay the same **for a year**:

- Buy a hardware reservation/commitment
- Specify the exact types of servers you need
- How much CPU/memory power they need
- Where they're physically located

Deck 5 p9



Reservations

Make a commitment for what you'll use for a year

Can reserve a VM, storage, backup space, etc.

Done entirely in the portal (or with API commands):
no need to talk to a salesperson or sign a contract

You get billed whether you use the VM or not

If you *are* using a VM that matches your reservation, the reservation is automatically applied to it

Not tied to specific VMs: can move 'em around

Deck 5 p10



To illustrate cost savings

Amazon RDS SQL Server Standard Edition

r6i.2xlarge: 8 cores, 64GB RAM

Multi-AZ for failover across Availability Zones

Includes SQL Server licensing, but not storage

Deck 5 p11



Reservations/commitments

	Cost for 1 Year	Savings	Cost Up Front
By the hour	\$53,260	-	\$0
Reserved, 1 year, no upfront payment	\$50,331	2%	\$0
Reserved, 1 year, partial upfront	\$47,934	10%	\$23,967
Reserved, 1 year, full upfront	\$46,976	12%	\$46,976
Reserved, 3 years, partial upfront	\$42,608	20%	\$63,913
Reserved, 3 years, full upfront	\$41,756	22%	\$125,269

Deck 5 p12



Reservations/commitments

People love saving money, sure.

But it doesn't make sense
to pay up front until you know
how your app will perform
on this hardware.

Savings	Cost Up Front
-	\$0
2%	\$0
10%	\$23,967
12%	\$46,976
20%	\$63,913
22%	\$125,269

Deck 5 p13



How Elliot's Enterprise does reservations

Dozens of SQL Servers,
1000s of databases, many teams

Looks at cloud bills overall

Buys reservations for long term use

Moves them around between
departments depending on server
usage, sizing

Deck 5 p14





How Alex the Architect does reservations

App code keeps growing, changing, tuning

User base keeps growing

Can't really predict more than a year out

Often can't afford to pay a year up-front

Does 1-year reservation, max, with no upfront

Deck 5 p15



Billie the Small Business Bird

3-4 SQL Servers, a few dozen databases,
most of the databases are third party

Apps don't change much, user counts stable

Business may be willing to do longer-term
reservations with more up front to save money

But we gotta get the reservation size right:
we can't use an extra SQL Server reservation
for other departments

Deck 5 p16



In all of the cases...

We can't guess the right reservation size up front.

We need to go live first,
then measure the server over time,
adjust its size,
and then settle on a reservation.

Deck 5 p17



The go-live plan

Go live a little oversized.

When you lift-and-shift to the cloud,
start with the **right instance type**
based on your backup/restore time goals.

But use a **slightly bigger instance size**
with more CPU & RAM than you think you need.

You're going to back it down over time.

Deck 5 p19



If you run your own VMs (IaaS)

If you use Availability Groups or mirroring:

- You'll have 2 or more servers
- They need Win or SQL patches every 30-90 days
- You're going to take an outage anyway

So leading up to that outage:

- Monitor performance on the primary
- Decide whether you need to upsize/downsize it
- Decide the new size you'll use

Deck 5 p20



On patch day, do rolling patches

Take the passive secondary replica, fully patch it

When it needs to be rebooted for the patch:

- Shut down Windows
- Change the VM's size (CPU count, memory)
- Turn it back on

Fail over to this node, and it's the new primary

Patch the former primary, shut it down, change size

Deck 5 p21



You could automate this

A client stores their server list in a table, including:

- Cluster name
- Node name
- Desired SQL Server patch level
- Desired instance size (CPU & memory)
- Desired location (data center, AZ)

And then built an app to compare each node's current state versus its desired state, plus apply changes.

Deck 5 p22



If you use IaaS w/o AGs or mirrors

If you run single SQL Servers without HA or DR:

- You're probably comfortable with some outages
- You probably have maintenance windows on weekends or after hours
- Just patch the instance, shut it down, change its size, and bring it back up

Deck 5 p23



If you use PaaS

If you use Azure SQL DB or Amazon RDS:

- Patching is already handled for you
- Just go into the portal and change the size

Yes, there will be an outage, possibly minutes long.

There is no Service Level Agreement for this.

Deck 5 p24



Deciding what to adjust

From the beginning of the day

1. Total data size in GB/TB, number of databases
2. Batch Requests per second
3. Wait time per core over time (like per second)
4. Is the same performance a success or a failure?
5. How much growth do we expect this year?
6. Can we just use Azure SQL DB features?



Using this data to make decisions

Are users happy with performance?

If yes, take the instance size down a notch
(you heard me)

If no, what's your top wait type?

- How do you tune for that wait type?
- If the most cost-effective way is adding hardware, do that – but otherwise, don't

